

27/03/2026

Introduction

IoT refers to a system with embedded devices collecting data with their sensors, exchanging information via networks, and, in many cases, controlling the physical outputs dependent on user input or environmental changes. Essentially, this means products must have hardware, software, and networking integrated into one system to sense, process, visualize, and, in some cases, actuate all in real time. This purpose of the EE3638 assignment is to develop an example for this system by designing a remotely operated environmental monitoring device that utilizes Raspberry Pi, a DHT11 digital temperature and humidity sensor, a web server, and can be remotely accessed via mobile devices [1].

The project for this course was to design a Raspberry Pi-based Environmental Monitoring Device functional as an environmental surveillance device by creating a Raspberry Pi 4-based Remote Environmental Monitoring Device that utilized DHT11 sensor technology as the main input to the Pi. The Raspberry Pi will take readings from the DHTSensor via GPIO and log the readings with timestamps in a CSV file and serve the logged results of sensor readings via a Flask-based web server to provide users access to the latest readings of the humidity and temperature sensor at any time and for any recent readings and the historical activity of both sensors through a line chart. To give more functionality to the device, an LED was integrated with the Raspberry Pi and can be turned on/off from the dashboard at any time. The web server and logging processes have been set up to launch on boot automatically with systemd providing reliability both during demonstration and in everyday use.

The primary project objectives are to read temperature and humidity digitally via Raspberry Pi; store those readings for analysis over time; distribute them through a web-based interface; allow remote monitoring from a smartphone; provide one additional extension feature deemed useful; ensure that the complete application runs reliably whilst being demonstrated. These objectives align directly with the assignment brief and its focus on the transfer of information, development of an application/software, acquisition of sensor data, analysis and visualisation of that data, and the ability to be accessed remotely as an IoT application.

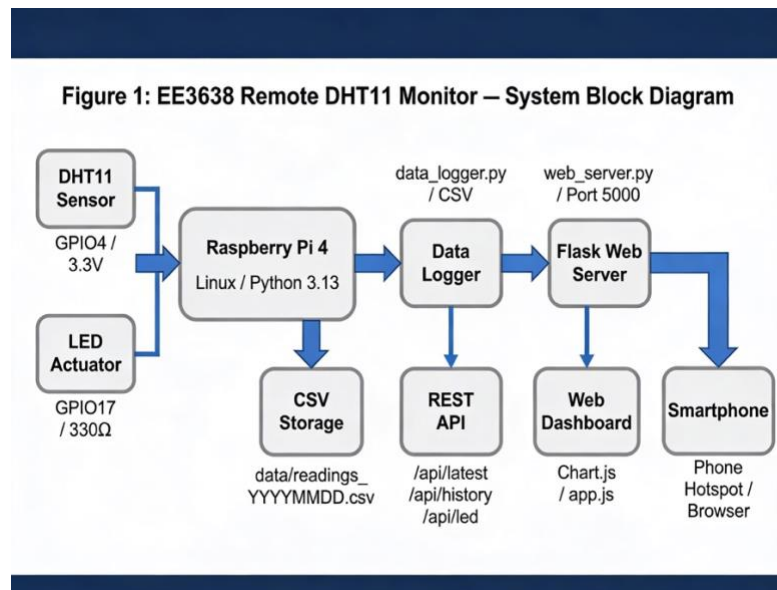


Figure 1: System Block Diagram

Requirements and Design Goals

Core functionality is derived from the assignment brief and consists of: 1) Using a Raspberry Pi microcontroller to collect real world data using a digital sensor, 2) Creating a web server to display that data on a webpage, 3) Having that data accessible remotely from a mobile device, and in real time, via wireless or mobile connections, and 4) Demonstrating the system live and producing a short engineering report that documents all aspects of the project. The specifications of the assignment indicate that additional functionality would also yield higher marks therefore several enhancements were implemented beyond the minimum requirements including: the ability to actuate LEDs, create a historical chart of data, add structured logging, and have automatic start-up after a reboot.

Several non-functional goals were identified along with the functional requirements of the system. The system must be easy to use, resist temporary loss of sensor readings, be able to display data using either a Raspberry Pi or a mobile device, and be modular in a way that the structure can be described clearly in the report. A sampling rate of five seconds was chosen as an appropriate compromise between being responsive while still being stable. The five seconds falls well within the specifications of a DHT11 sensor, while also allowing the user to have a "live" experience when navigating through the dashboard without overloading either the sensor or the associated software stack. In addition, the design intent is to separate the four modules involved in sensing, storing, presenting, and actuating such that they are all totally independent of one another, and can therefore be tested and described very easily.

Requirement (from brief)	Implementation	Evidence
Read digital sensor on Raspberry Pi	DHT11 on GPIO4 via dht_reader.py	Fig 17
Log readings with timestamps	data_logger.py writes readings_YYYYMMDD.csv every 5 s	Fig 10
Create a web server	Flask app in web_server.py on port 5000	Fig 12
Access readings remotely from mobile device	Phone browser over hotspot at <a href="http://<pi-ip>:5000/">http://<pi-ip>:5000/	Fig 13
Wireless/mobile transmission path	Smartphone hotspot LAN	Fig 4
Additional functionality (actuator)	LED on GPIO17	Fig 18
Additional functionality (data visualisation)	Chart.js dual-axis line graph from /api/history	Fig 14

Hardware Design

Raspberry Pi 4 was used as the hardware platform for the project, while the assignment brief also refers to learning about the Raspberry Pi and Raspberry Pi Pico, the Pi 4 was the better option for this project due to its ability to run a full Linux operating system, create Python websites, and have local Web browsing/debugging capabilities as well as have direct GPIO access for connecting digital sensors and actuators. The primary sensor used was a DHT11 module, chosen because it directly measures both temperature and humidity and outputs this data in digital form, matching the brief's emphasis on digital sensor reading. An LED and resistor were added as a simple actuator to demonstrate two-way interaction rather than sensing alone.

Using three wires, a DHT11 was connected; a signal wire was connected to BCM GPIO4 (pin 7) [2] of the Raspberry Pi header, and a 3.3V power wire was connected to pin 1, while the ground wire was connected to pin 6. In this way, the logic level (3.3V) was safe for a direct connection to the GPIO interface [3]. DHT11s cannot be run from 5V sources. BCM

GPIO4 was used because it was the most common GPIO pin used in DHT11 documentation and example codes. The layout information for these devices is contained in the system configuration file.

An LED actuator was connected to the BCM GPIO17 by way of a 330 Ω resistor in series, with the common return path being ground. The GPIO17 output was utilized as it is a standard general-purpose output pin and had no issues with the GPIO library being used in the LED actuator control module. The resistor helped to protect both the LED and the GPIO by limiting current flow through each device. Although there was little hardware complexity added with this device, it adds significant value in terms of assessment by showing that the project can both sense and control events as well as just display information.

The physical circuit was laid out on a breadboard using jumper wires for interconnections. A breakout AdaFruit T-Cobbler was used to make verification of the header pin mappings much easier. This improved both the reliability and clarity during debugging and troubleshooting. An intentional effort was made from a mechanical design point of view to keep the final physical design simple so as to allow the focus of everybody involved to remain on quality and reliability of the power supplies as well as the structure and functionality of the software being used to control/input the LED actuator.

Component	Module Pin	Pi Physical Pin	BCM GPIO	Function
DHT11	S (signal)	7	GPIO4	Digital data
DHT11	+ (VCC)	1	3V3	Power supply
DHT11	– (GND)	6	GND	Common ground
LED	Anode	11	GPIO17	Actuator output
LED	Cathode	9	GND	Common ground
Resistor (330 Ω)	In series with LED	—	—	Current limiting

Figure 2: Wiring Table

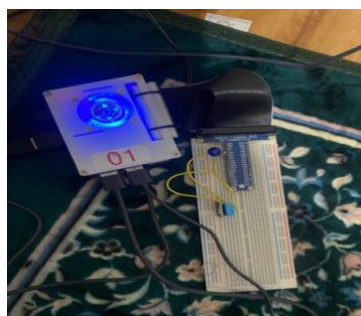


Figure 3: Hardware Design

Network Design

One of the main practical barriers to a successful project is the network path from the Raspberry Pi to the mobile device. As the project has a requirement for a wireless or mobile component between the source and destination, and also notes that students may face difficulties with the Brunel network, in practice, the use of a direct device-to-device connection over Eduroam would not be reliable enough for a clear demonstration. In order to resolve this, a smartphone mobile hotspot is used as a private access point (both the Raspberry Pi and mobile phone were connected to the same smartphone hotspot), which satisfied the project requirement for a wireless/mobile transmission path, and created an environment for a stable and repeatable demonstration.

The Raspberry Pi hosted a Flask server (on port 5000) and exposed the dashboard at <http://pi-ip:5000/> – once both devices were connected to the smartphone mobile hotspot, the mobile phone browser could connect to the Raspberry Pi via the Pi's IP address and retrieve both the HTML user interface and JSON API responses produced by the Flask server on the Raspberry Pi. This approach ensures that the deployment was kept simple, that there were no other dependencies such as tunnelling or port forwarding, and that the system could be easily reproduced to demonstrate the installation of the system. Thus, the design of the network was a conscious engineering trade-off: to give priority to reliable and clear networking rather than to introduce unnecessary complexity with networking.

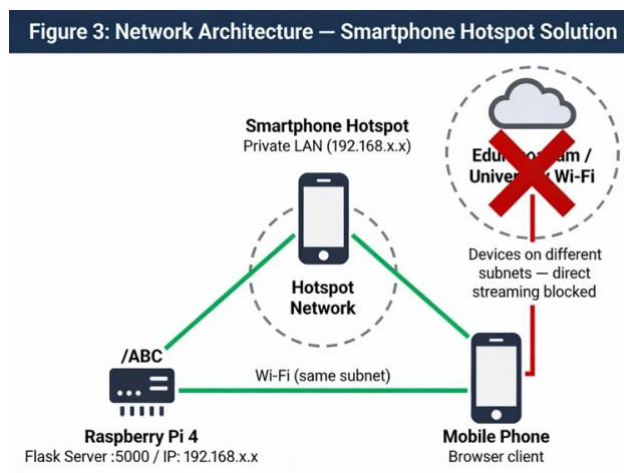


Figure 4: Network Diagram

Software Architecture

Software exists as two independent processes with each managed as an individual systemd service. The first being `ee3638_logger.service`; controlling the running of `data_logger.py`; which has the only responsibility of reading the DHT11 sensor and saving readings into a daily CSV file. The second being `ee3638_web.service`; controlling the

running of `web_server.py`; which has the only responsibility of serving data from those CSV files to the web clients.

The primary reason for splitting the application into two processes was to avoid runtime contention errors that would otherwise occur when multiple processes attempted to access the DHT11 driver, which uses a PulseIn kernel level message queue. By assigning ownership of the DHT11 sensor to the logger process, contention errors were eliminated so that the web server remains non-blocking and responsive.

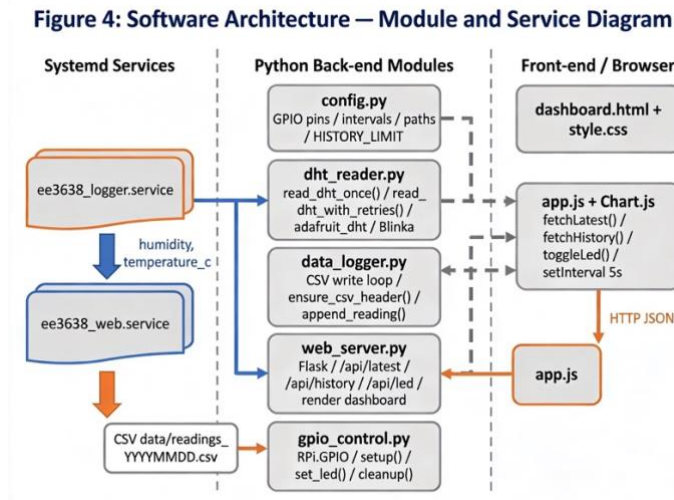


Figure 5: Software Architecture

The information proceeds through a sequence of stages within the system. A pulse train is created from digital data generated from the DHT11 via GPIO4. The Digital Pulse Train is translated into usable DHT11 formatted data via Adafruit's CircuitPython DHT Library within `dht_reader.py`. The digital data is sent via `data_logger.py`, which converts the ISO timestamp formatted digital data into a CSV formatted record and then writes it to the `data/readings_YYYYMMDD.csv`. The `web_server.py` reads the file and returns it via JSON when an API request is received from the browser via `app.js`, which repeatedly queries the API once every five seconds to update both the DOM elements and the data structure utilized by `Chart.js`.

The application consists of six Python/JavaScript source files and one central configuration file. Their responsibilities are described below:

- The central configuration file for this application is `config.py`. The file contains all the runtime/configurations in one place, so changes to GPIO pins, timing intervals, path to files and limit of history records will only need to be made in this one location. The main configuration parameters within `config.py` are:
`DHT_SENSOR_TYPE = "DHT11"`, `DHT_GPIO_PIN = 4`, `READ_INTERVAL_SECONDS =`

5, DHT_MAX_FAIL_SECONDS = 10, DATA_DIRECTORY = "data", LOG_FILE = "logs/app.log", HISTORY_LIMIT = 50. The five second polling rate was used since it provides an optimum level of responsiveness and stability. It is significantly greater than the DHT11 minimum sampling period of one second, providing the user with a nearly real-time experience on the dashboard.

```
DHT_SENSOR_TYPE = "DHT11"      # sensor model
DHT_GPIO_PIN = 4                # BCM numbering, GPIO4 (physical pin 7)

READ_INTERVAL_SECONDS = 5      # main loop poll interval (respects >=1 s DHT11 limit)
MAX_READ_RETRIES = 5           # retries per measurement
RETRY_DELAY_SECONDS = 2        # delay between retries
DHT_MAX_FAIL_SECONDS = 10

DATA_DIR = "data"
LOG_DIR = "logs"
LOG_FILE = "logs/app.log"

HISTORY_LIMIT = 50              # number of recent readings for /api/history
```

Figure 6: Config.py configuration

- The entire interaction with the physical DHT11 sensor is done through the dht_reader.py file. When this module is imported, a single instance of adafruit_dht.DHT11 is created and made available through the board.D4 (BCM GPIO4) pin, ensuring that only one driver instance is created during the lifetime of an app. The read_dht_once() function is used to attempt to read once and returns either (humidity, temperature_c) if successful or (None, None) if a RuntimeError occurred (the most common mode of failure with DHT11 sensors is a checksum error). The read_dht_with_retries() function implements the retry feature by calling read_dht_once() until a valid reading has been made or until the specified DHT_MAX_FAIL_SECONDS (10 seconds) has been exceeded. Each time a call to read_dht_once() fails, it will log the error at WARNING level, while every successful call will log at the INFO log level. The retry mechanism helps to prevent loss of data from having one bad reading, as the DHT11 sensor has the potential to produce bad readings intermittently [4], [5].

```
def read_dht_once() -> Tuple[Optional[float], Optional[float]]:
    """
    Perform a single DHT read.
    Returns (humidity, temperature_c) or (None, None) if invalid.
    """
    try:
        temperature_c = dht_device.temperature
        humidity = dht_device.humidity
        if humidity is None or temperature_c is None:
            return None, None
        return float(humidity), float(temperature_c)
```

Figure 7: dht_reader.py

- `data_logger.py` acts as both a sampling loop and a persistence loop. It logs a configured interval to determine what time the loop started, and then begins an infinite while statement that is `True`. At the start of each iteration of this loop:
 - ❖ 1. It uses `current_csv_path()` to determine today's CSV file path (which will be formatted as `readings_YYYYMMDD.csv`); so that there will be a new CSV every day
 - ❖ 2. It calls `ensure_csv_header()` to create the CSV file (with the correct header row) if it does not exist
 - ❖ 3. It collects a timestamp in the current ISO format
 - ❖ 4. It gets the temperature readings and humidity readings using `read_dht_with_retries()` (10 retries, 2 seconds between each retry)
 - ❖ 5. If the readings are valid it will write the row into the CSV file; If the reading returned (`None, None`), then it will do nothing except log a `WARNING` message to state that a reading failed
 - ❖ 6. It sleeps for `READ_INTERVAL_SECONDS`; in this way the `data_logger.py` will only ever have validated readings in the CSV.
- The program `web_server.py` is a Flask program providing three types of content to your users, a REST API, an HTML Dashboard and LED control routes to access LED controls. The program calls `gpio_setup()` at startup to initialize GPIO17 configured as an OUT. Two helper functions access our source of data, the CSV file; `get_latest_from_csv()` will read today's file, locating the last record and returning it as a dictionary; `get_history_from_csv(limit)` will load the entire file, locating the last `HISTORY_LIMIT` (in this case 50) records, and returning these records as a JSON array. The API call to `/api/latest` will call `get_latest_from_csv` and return the most current data as a JSON object containing status, source, and data (in this case `timestamp_iso`, `temperature_c`, `humidity_percent`). The API call to `/api/history` will call `get_history_from_csv` and return the records structured as in the previous example as a JSON array. The API call to `/` will cause the `web_server.py` program to render the `dashboard.html` page via its use of Flask's Jinja2 template rendering. The handling of LED's is accomplished via: API call to `/api/led` GET (to provide the current state) and to `/api/led` POST (to process the incoming request of `{"on": true|false }`), call `set_led()` to set the LEDs lights using GPIO, and save the state of the LED light in `app.config["LED_STATE"]`). All API calls (GET and POST) will produce

structured logs at INFO level to logs/web.log [8].

```
@app.route("/api/latest")
def api_latest():
    """
    Return latest reading from CSV only.
    Status:
    - ok: if today's CSV has at least one row
    - error: if no data is available
    """
    logger.info("GET /api/latest")

    latest = get_latest_from_csv()
    if not latest:
        return jsonify({"status": "error", "source": "none", "data": None})

    data = {
        "timestamp_iso": latest["timestamp_iso"],
        "temperature_c": latest["temperature_c"],
        "humidity_percent": latest["humidity_percent"],
    }
    # For simplicity, treat recent CSV as live
    return jsonify({"status": "ok", "source": "csv", "data": data})
```

Figure 8: api/latest program code

- ❖ app.js is responsible for controlling the front end of the application. When the app loads, it executes tick() immediately after receiving the DOM onload event (DOMContentLoaded) and will call both tick() and fetchLedStatus() every five seconds using setInterval.
- 1) The tick() method runs fetchLatest() and fetchHistory() concurrently.
- 2) fetchLatest() calls the api on /api/latest to retrieve sensor data and updates status, timestamp, temperature, and humidity on the front end. If the api returns an error, a red warning will display stating that the sensor data is not OK.
- 3) fetchHistory() calls /api/history to receive previous sensor data, rebuilds the table with that past data, and updates the chart as well. For each timestamp, it extracts the time portion and converts the value to a number for display purposes.
- 4) When the chart is created, it is a new Chart.js line graph with two axes (temperature on the left, humidity on the right). Where the chart already exists, only the data is updated to allow smooth refreshing of the chart with the new data [6].
- 5) The ToggleLed() method checks whether the led is currently on/off, sends the opposite value to the controller via /api/led, then updates the front end with any response received from the controller.

Implementation and Operation

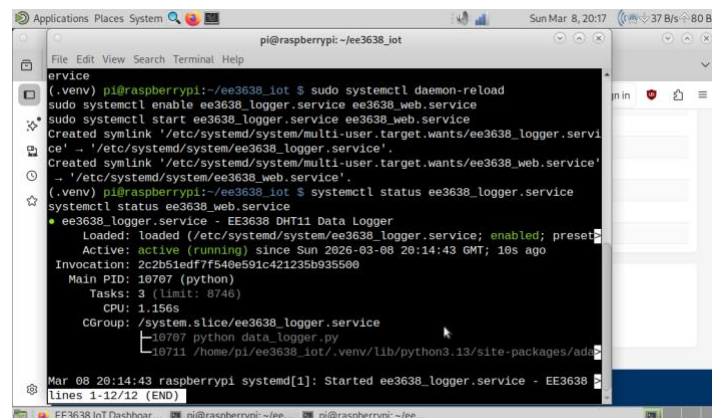
The RPi utilized an operating system using Debian GNU/Linux 13 to build its application using the programming language of Python 3.13. In recent versions of Debian, there are stricter policies on how to manage the system's "global" or "default" Python environment

than there were previously, so projects utilizing Python only implement a "virtual environment" (.venv) for the installation of libraries specific to the project.

The libraries utilized for this project comprised mostly of Flask (for the web application) and Adafruit's `DHT` Library (used to read data from the DHT sensor). The combination of the above platform stack options provided a fully stand-alone implementation of the application without any ability to have conflicting dependencies.

The avoidance of multiple processes reading from the DHT11 directly at the same time was also an important design decision. The DHT11 is sensitive to timing issues and tests conducted previously produced runtime warnings intermittently. The logger was responsible for performing regular sensor sampling, while the web server obtained information through the logger's CSV output for both current and historical data. This reduced the risk of contention between the two and allowed for more predictable results. From the user's perspective, all the web server was doing was presenting the last CSV that was written to, and, therefore, providing "live" data. This was sufficient for the dashboard refresh interval specified in the coursework.

To deploy the system, two scripts were created that launch the processes, run_logger.sh and run_web.sh. Each script activates the virtual environment, runs the appropriate python script (data_logger.py for run_logger.sh and web_server.py for run_web.sh) and was used as a systemd service to allow automatic startup of the complete system after reboot. Compared to launching the processes from the command line manually, this made the time for setup during demo much shorter and greatly increased reliability in the case the system restarted unexpectedly [9].



```
pi@raspberrypi: ~/ee3638_iot
service
(.venv) pi@raspberrypi:~/ee3638_iot $ sudo systemctl daemon-reload
sudo systemctl enable ee3638_logger.service ee3638_web.service
sudo systemctl start ee3638_logger.service ee3638_web.service
Created symlink '/etc/systemd/system/multi-user.target.wants/ee3638_logger.service' → '/etc/systemd/system/ee3638_logger.service'.
Created symlink '/etc/systemd/system/multi-user.target.wants/ee3638_web.service' → '/etc/systemd/system/ee3638_web.service'.
(.venv) pi@raspberrypi:~/ee3638_iot $ systemctl status ee3638_logger.service
systemctl status ee3638_logger.service
● ee3638_logger.service - EE3638 DHT11 Data Logger
   Loaded: loaded (/etc/systemd/system/ee3638_logger.service; enabled; preset=
   Active: active (running) since Sun 2020-03-08 20:14:43 GMT; 19s ago
   Invocation: 2c2b51edf7f540e591c421235b935500
   Main PID: 10707 (python)
   Tasks: 3 (limit: 8746)
   CPU: 1.156s
   CGroup: /system.slice/ee3638_logger.service
           └─10707 python data_logger.py
             └─10711 /home/pi/ee3638_iot/.venv/lib/python3.13/site-packages/ad

Mar 08 20:14:43 raspberrypi systemd[1]: Started ee3638_logger.service - EE3638
Lines 1-12/12 (END)
```

Figure 9: Systemctl Status

The normal operation of the overall system is simple. Following system boot, the logger service starts writing sensor readings for every 5 seconds into the appropriate CSV file for the day, while at the same time, the web service publishes the web-based dashboard and

APIs available at port 5000, and any browser on either the Pi or the phone would be able to connect to the web-based dashboard to review current sensor data, recent history, output data in chart format, and also control the state of the LEDs located in the sensor section of the electronics enclosure. Therefore, there is an automatic flow of information from the actual acquisition of sensors to the end user for decision-making once the overall system has been powered on.

Figure 10: CSV sample

readings_20260308		
timestamp_iso	temperature_c	humidity_percent
2026-03-08T18:24:35	18.8	63.0
2026-03-08T18:24:40	18.6	62.0
2026-03-08T18:24:46	18.6	62.0
2026-03-08T18:24:51	18.6	62.0
2026-03-08T18:24:59	18.6	62.0
2026-03-08T18:25:04	18.6	62.0
2026-03-08T18:25:09	18.6	62.0
2026-03-08T19:01:17	18.6	62.0
2026-03-08T19:01:22	18.6	62.0
2026-03-08T19:01:27	18.7	69.0
2026-03-08T19:01:35	18.8	70.0
2026-03-08T19:01:40	18.9	72.0
2026-03-08T19:01:45	19.2	76.0
2026-03-08T19:01:50	20.3	81.0
2026-03-08T19:01:56	21.3	85.0
2026-03-08T19:02:01	22.1	85.0

Figure 6: CSV sample

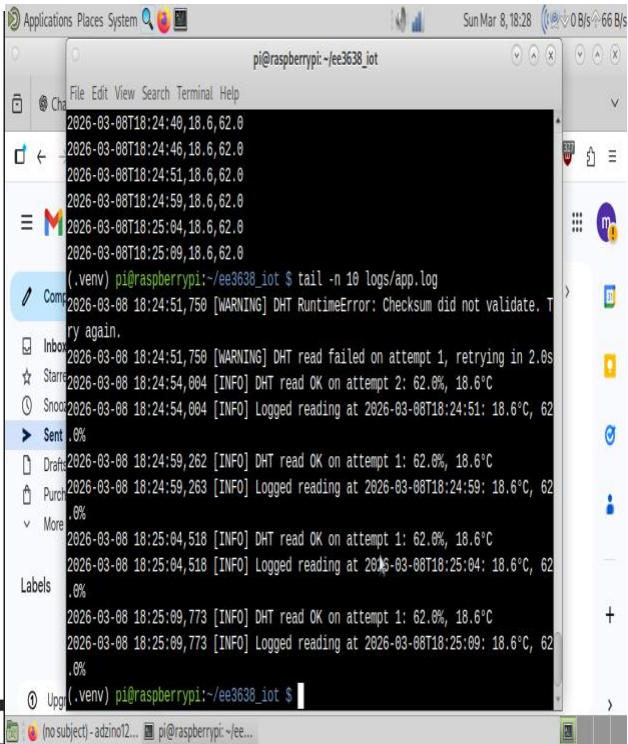


Figure 11: DHT Log excerpt

Testing & Evaluation

The testing of the system occurred over a long period of time and in several phases. The first phase of testing determined whether or not the individual subsystems functioned independently before they were combined to form the complete system. In the first phase, an independent sensor reader was able to read the data from the DHT11 sensor and display the results on the terminal as part of a complete test. In the second phase the ability of the logger to store multiple readings every 5 seconds to a CSV file was confirmed to meet specification. The third and fourth phases validated the data returned from the Flask REST API and browser dashboard were both accurate via the API calls /api/latest and /api/history and that the data was displayed on the dashboard correctly. In the fifth phase, it was confirmed that mobile clients that were connected to the same hotspot as the browser could access the dashboard page. The sixth phase of the testing process

confirmed that the LED control route and buttons were able to turn the physical actuator on and off using the web command processed through the Flask REST API. The above described staged approach to testing is aligned with the engineering principle of verifying components could operate independently from one another prior to performing full integration test of the entire system.

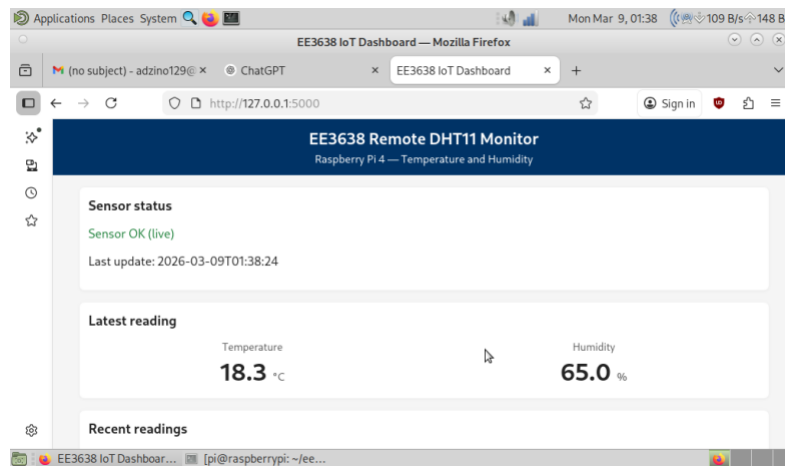


Figure 12: Dashboard on Pi

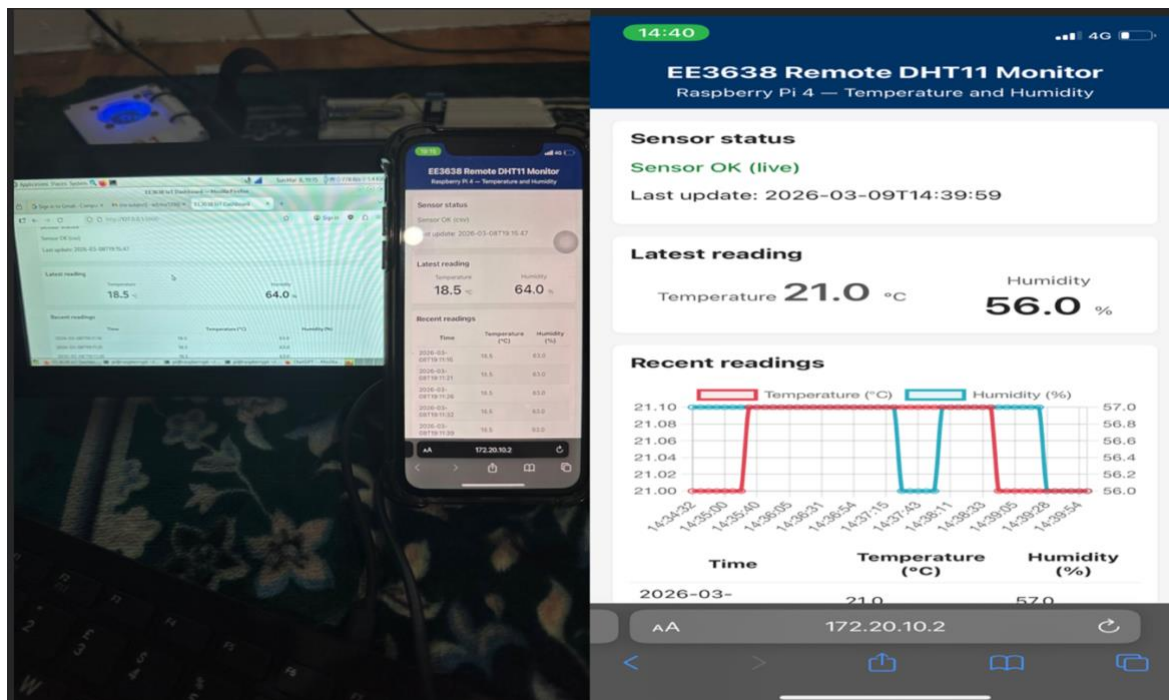


Figure 13: Dashboard observation on phone

The data logger performed consistently, producing a timestamped CSV record at approximately five-second intervals, as defined in the configuration. This made the historical record both easy to inspect manually and suitable for plotting in the dashboard.

The use of an ISO timestamp format also simplified ordering and readability. From an evaluation perspective, this showed that the data collection pipeline was stable enough to support real-time presentation and post-run analysis.

The dashboard showed the success of end-to-end processing. The latest reading section was continuously updated with each new value that was logged; and recent readings displayed both a table and graph (via the history API). In this way, the user had more than just a static readout; they had a visual representation of how the current value had changed over time. For example, the graph was very helpful in reflecting the small changes to temperature and humidity over time that would not be as easily seen from a single current reading. This shows that the system is effective at meeting the learning objective based on the collection, analysis, and visualization of sensor data.

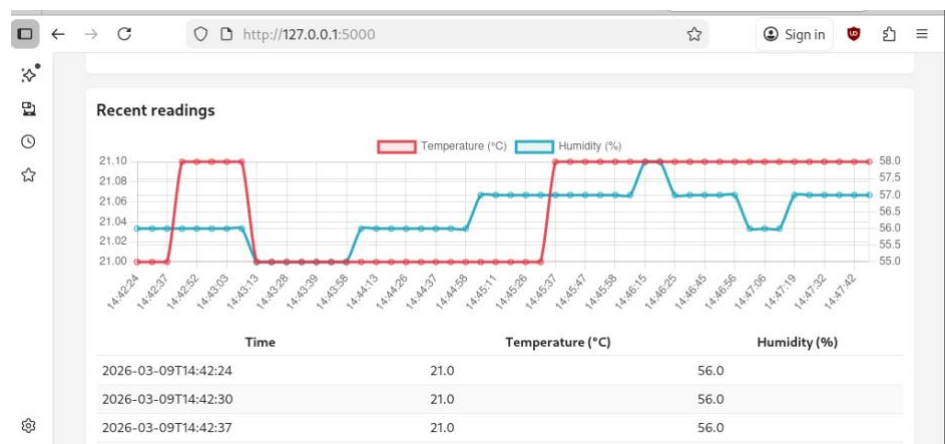


Figure 14: Chart.js graph

The system has also shown reliable operation working successfully. The use of a retry-based sensor reader allows the DHT errors to be transient in nature, thus preventing immediate failure. Due to invalid readings being skipped rather than written to the file, the logger continued to function after a single read failure. Additionally, since both the logger and web server were using systemd, both were able to start automatically after a reboot. As a result of this, both the demonstration readiness and the value of the system in longer-term deployments will be greatly enhanced.

Nonetheless, there are still some limitations to this sensor. It is an affordable sensor meant for educational use only therefore, it is not very accurate when measuring the environment's conditions. During earlier tests it created intermittent timing and checksum errors, evidenced by the log file output. To overcome these limitations while developing this project, the program implemented retries, used a slower sample rate and allowed for the use of the CSV-backed presentation method rather than directly polling the multiple

processes of the sensor system. Given the needs of the project, these limitations were deemed reasonable and well warranted trade-offs of simplicity vs reliability.

	Test	Expected result	Actual result	Evidence
1	Manual ./dht_reader.py (services stopped)	Valid humidity + temperature printed	Pass — 62.0%, 18.5°C returned	Fig 17
2	./data_logger.py runs, CSV grows at 5 s intervals	New timestamped rows appended	Pass — confirmed in CSV	Fig 16, Fig 11
3	curl /api/latest returns JSON	status:"ok", correct fields	Pass — JSON returned immediately	Fig 15
4	curl /api/history returns 50 rows	Array of 50 readings	Pass — count:50 confirmed	Fig 15
5	Dashboard on Pi (Firefox)	Status, values, chart and table visible	Pass	Fig 12
6	Dashboard on phone over hotspot	Same dashboard accessible	Pass	Fig 13
7	LED toggle from dashboard	LED turns on/off, state reflected in UI	Pass	Fig 18 (LED status visible)
8	Reboot → services auto-start	Both services active without manual launch	Pass — active (running) shown	Fig 9
9	Quantitative Evaluation	Mean temperature and humidity with standard deviation	Pass	Fig 19


```
pi@raspberrypi:~/ee3638_iot $ cd ~/ee3638_iot
nano web_server.py
pi@raspberrypi:~/ee3638_iot $ sudo systemctl restart ee3638_web.service
pi@raspberrypi:~/ee3638_iot $ sudo systemctl restart ee3638_logger.service
pi@raspberrypi:~/ee3638_iot $ cd ~/ee3638_iot
source .venv/bin/activate
curl http://127.0.0.1:5000/api/latest
curl http://127.0.0.1:5000/api/history
{"data":{"humidity_percent":57.0,"temperature_c":21.1,"timestamp_iso":2026-03-09T14:38:28},"source":"csv","status":"ok"}
{"count":50,"data":[{"humidity_percent":56.0,"temperature_c":21.1,"timestamp_iso":2026-03-09T14:32:58}, {"humidity_percent":56.0,"temperature_c":21.1,"timestamp_iso":2026-03-09T14:33:03}, {"humidity_percent":56.0,"temperature_c":21.1,"timestamp_iso":2026-03-09T14:33:08}, {"humidity_percent":56.0,"temperature_c":21.1,"timestamp_iso":2026-03-09T14:33:14}, {"humidity_percent":56.0,"temperature_c":21.1,"timestamp_iso":2026-03-09T14:33:21}, {"humidity_percent":56.0,"temperature_c":21.1,"timestamp_iso":2026-03-09T14:33:26}, {"humidity_percent":56.0,"temperature_c":21.1,"timestamp_iso":2026-03-09T14:33:32}, {"humidity_percent":56.0,"temperature_c":21.1,"timestamp_iso":2026-03-09T14:33:37}, {"humidity_percent":56.0,"temperature_c":21.1,"timestamp_iso":2026-03-09T14:33:42}, {"humidity_percent":56.0,"temperature_c":21.1,"timestamp_iso":2026-03-09T14:33:47}, {"humidity_percent":56.0,"temperature_c":21.1,"timestamp_iso":2026-03-09T14:33:53}, {"humidity_percent":56.0,"temperature_c":21.1,"timestamp_iso":2026-03-09T14:33:58}, {"humidity_percent":56.0,"temperature_c":21.1,"timestamp_iso":2026-03-09T14:34:03}, {"humidity_percent":56.0,"temperature_c":21.0,"timestamp_iso":2026-03-09T14:34:08}, {"humidity_percent":57.0,"temperature_c":21.0,"timestamp_iso":2026-03-09T14:38:28}]}
pi@raspberrypi:~/ee3638_iot $
```

Figure 15: API Curl output

```
pi@raspberrypi:~/ee3638_iot
received SIGINT
Traceback (most recent call last):
  File "/home/pi/ee3638_iot/./data_logger.py", line 70, in <module>
    main()
  File "/home/pi/ee3638_iot/./data_logger.py", line 72, in main
    print(f'{timestamp_iso} {temperature_c:.1f}°C {humidity:.1f}%')
KeyboardInterrupt
(.venv) pi@raspberrypi:~/ee3638_iot $ ls data
cat data/readings_$(date +%Y%m%d).csv
readings_20260308.csv
timestamp_iso,temperature_c,humidity_percent
2026-03-08T18:24:35,18.8,63.0
2026-03-08T18:24:40,18.6,62.0
2026-03-08T18:24:46,18.6,62.0
2026-03-08T18:24:51,18.6,62.0
2026-03-08T18:24:59,18.6,62.0
2026-03-08T18:25:04,18.6,62.0
2026-03-08T18:25:09,18.6,62.0
(.venv) pi@raspberrypi:~/ee3638_iot $
```

Figure 16: data_logger.py running

```
pi@raspberrypi:~/ee3638_iot
python3.5 pyusb-1.3.1 rpi_ws281x-5.0.0 sysv_ipc-1.2.0 typing_extensions-4.15.0
(.venv) pi@raspberrypi:~/ee3638_iot $ python -c 'EOF'
import board
import adafruit_dht
print("board OK, adafruit_dht OK")
EOF
board OK, adafruit_dht OK
(.venv) pi@raspberrypi:~/ee3638_iot $ ^[[200~cd ~/ee3638_iot
bash: $'\E[200~cd': command not found
(.venv) pi@raspberrypi:~/ee3638_iot $ cd ~/ee3638_iot
nano config.py
(.venv) pi@raspberrypi:~/ee3638_iot $ nano dht_reader.py
(.venv) pi@raspberrypi:~/ee3638_iot $ chmod +x dht_reader.py
(.venv) pi@raspberrypi:~/ee3638_iot $ cd ~/ee3638_iot
source .venv/bin/activate
Humidity: 63.0% Temperature: 19.6°C
(.venv) pi@raspberrypi:~/ee3638_iot $ cd ~/ee3638_iot
source .venv/bin/activate
./dht_reader.py
Humidity: 84.0% Temperature: 29.7°C
(.venv) pi@raspberrypi:~/ee3638_iot $ scrot -d 5
```

Figure 17: dht_reader.py running snippet

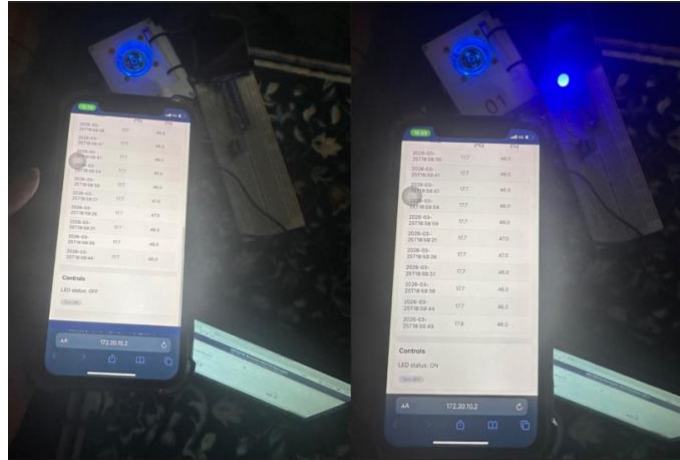


Figure 18: LED toggle from dashboard

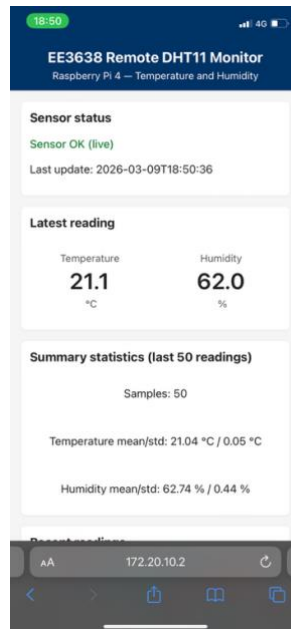


Figure 19: Quantitative Evaluation

Discussion and Limitations

All of the major objectives for this assignment have been completed (in terms of how they are being presented on the page). The setup is a Raspberry Pi that reads a digital sensor (DHT11), saves and displays environmental data, provides access to the output via a web interface, and permits remote access from a mobile device. In addition to meeting the minimum requirements, the following features have added value to the project: an LED actuator, graphical history of environmental data, more structured logging of the output, and systemd-based auto-start functionality; all of these aspects have resulted in improved quality of both the technical output of the project and the evidence of the demonstration.

The primary limitations of the current implementation are related to the quality of the DHT11 sensor and security for communicating data from the sensors to the Internet. While the DHT11 is sufficient for educational/prototyping purposes, its accuracy and speed are both limited compared to other commercially available sensors. Additionally, the CSV file used to store the measurement data does not provide a high level of scale over time when multiple sensors are deployed (and therefore have long-term storage) than if either a real database (such as SQLite or a time-series database).

The primary means of improving the current system in the future would be to replace the DHT11 sensor with either a BME280 or SHT series sensor (which provide better accuracy) and upgrade the way measurement data is stored from CSV files to SQLite (or a time-series database) to allow for better querying functionality and longer-term retention rates of measurement data. Both of these infrastructure upgrades will allow the project to function in a manner closer to that of an industrial IoT or production-level IoT platform.

Conclusion

A remote accessible IoT monitoring and control system has been designed and implemented, utilising a Raspberry Pi 4, DHT11 Sensor and an LED actuator. The resulting system reads digital temperature and humidity data, logs the data in structured CSV files, exposes data via a Flask web server, and displays the data in a responsive dashboard with graphical charting. Remote control of the LED is achieved from a smartphone via hotspot connection. All of these components were achieved and fulfil the major requirements of the EE3638 project brief and demonstrate the integration of hardware with Linux, Python, Networking, and web technologies as a fully functioning embedded system.

One major learning point from this project is that reliability is equally as important as basic functionality in IoT systems. Retry handling, structured logging, modular software design and automatically starting the services were all major contributors to converting the original simple sensor demonstration into a stable and assessable engineering system. Overall, the project achieved an excellent balance between technical simplicity, clarity of design and additional valuable functionality.

References:

- [1] Brunel University London, "EE3638 Design for IoT – Remotely controlled device individual assignment brief," Module handout, January 2026.
- [2] J. Cosmas, "Lab_02_IntroductionToLinuxPython-v4," EE3638 Lab Slides, Brunel University London, 2026.

- [3] R. Santos and S. Santos, "Raspberry Pi – DHT11 Temperature and Humidity Data Logger," Random Nerd Tutorials, [Online]. Available: <https://randomnerdtutorials.com/raspberry-pi-temperature-humidity-data-logger/> [Accessed: March 2026].
- [4] Adafruit Industries, "CircuitPython DHT Library," GitHub, [Online]. Available: https://github.com/adafruit/Adafruit_CircuitPython_DHT [Accessed: March 2026].
- [5] Adafruit Industries, "Adafruit Blinka – CircuitPython Libraries on Linux and Python," [Online]. Available: https://github.com/adafruit/Adafruit_Blinka [Accessed: March 2026].
- [6] Chart.js Contributors, "Chart.js – Simple yet Flexible JavaScript Charting Library," [Online]. Available: <https://www.chartjs.org/> [Accessed: March 2026].
- [7] D-Robotics, "DHT11 Temperature and Humidity Sensor Datasheet," [Online]. Available: <https://www.mouser.com/datasheet/2/758/DHT11-Technical-Data-Sheet-Translated-Version-1143054.pdf> [Accessed: March 2026].
- [8] Pallets Projects, "Flask – Web Development One Drop at a Time," [Online]. Available: <https://flask.palletsprojects.com/> [Accessed: March 2026].
- [9] freedesktop.org, "systemd.service – Service unit configuration," systemd Documentation, [Online]. Available: <https://www.freedesktop.org/software/systemd/man/systemd.service.html> [Accessed: March 2026].